Self-Learning Plan: React Native + Expo in 4 Weeks

20 study days inside 30 calendar days · self-paced, no teacher · level: easy to medium.

Each week has two parts:

1. **Part A: Learn (Monday–Tuesday).** The concepts you need for the week, each with an article or documentation page to read, and where to see it in this app.
2. **Part B: Tasks (Wednesday–Friday).** 4 tasks that use what you just learned. Each task adds a real feature to this app and says clearly **what the objective is** and **what to do**.

The weekly schedule

Day	Part	What you do	Time
Monday	A: Learn	First half of the week's concepts	~4 h
Tuesday	A: Learn	Second half of the concepts + "Check yourself" questions	~4 h
Wednesday	B: Tasks	Task 1 and Task 2 (the smaller ones)	~4 h
Thursday	B: Tasks	Task 3	~4 h
Friday	B: Tasks	Task 4 + the weekly check	~4 h
Saturday	–	Optional: catch up on anything unfinished	–
Sunday	–	Rest	–

Calendar days 29–30 (after Week 4): record a short demo and fill in the self-check.

The 4 weeks at a glance

Week	Calendar days	Theme	Tasks
1	1–7	Screens and UI	Support row · Rating stars · Details card · Reviews screen
2	8–14	Data: API, React Query, forms	Category screen · Sort by price · Out-of-stock · Discount field
3	15–21	Build your own module	Users data · Users tab · User profile · Login
4	22–28	Polish and mini project	Offline banner · Home widget · Cart tab · Cart summary

How to learn and how to work

- **Learning:** read the links for each concept. You don't need to memorise anything. Then open the "See it in the app" file and find the concept in real code. Answer the "Check yourself" questions in your own words in `PROGRESS.md`.
- **Tasks:** one branch and one pull request (PR) per task, titled like `Task 2.3: out-of-stock action`. Run `npm run validate` before pushing.
- **When you're stuck:** re-read the task, open the file it points to (the pattern usually already exists), search the exact error message, and re-read the concept's links. If you use an AI assistant, ask it to **explain**, not to write the code. Still stuck after 45 minutes? Write the blocker in your PR and move on.

Setup (before Week 1)

- [ ] Install **Node.js** (LTS), **Git**, **VS Code** (ESLint, Prettier, Tailwind CSS IntelliSense extensions) and **Expo Go** on your phone (SDK 58).
- [ ] Create a **GitHub** account and **fork** this repository (how to fork).
- [ ] Clone your fork, `npm install`, `npm start`, scan the QR code with your phone (GETTING-STARTED.md).
- [ ] Learn how to open a PR from your fork (guide).
- [ ] Create `PROGRESS.md` in your fork (template at the end).

Cheat sheet: building screens in this app

See every component live in the app: **Settings → UI Sandbox**.

You need	Use
Text	`Text` from `@/shared/ui/text` (`variant="h3"`, `"large"`, `"muted"`)
Button	`Button` from `@/shared/ui/button` (`variant="outline"`, `"destructive"`)
Card / box	`className="rounded-lg border border-border bg-card p-4"`
Badge, Avatar, Input	`@/shared/ui/badge` · `@/shared/ui/avatar` · `@/shared/ui/input`
Icon	`Icon` from `@/shared/ui/icon` + an icon from lucide
Loading/error/empty	`LoadingState`, `ErrorState`, `EmptyState` from `@/shared/components/screen-states`

Colors: never hex (`#4f46e5`). Use theme classes so dark mode works by itself: `bg-background` · `bg-card` · `text-foreground` · `text-muted-foreground` · `text-primary` · `bg-primary` · `text-destructive` · `bg-success` · `border-border`.

Building…	Copy from
A list screen	`src/modules/catalog/features/product-list/product-list-screen.tsx`
A details screen	`src/modules/catalog/features/product-detail/product-detail-screen.tsx`
A form	`src/modules/catalog/components/product-form.tsx`
A settings-style list	`src/modules/settings/features/app-settings/settings-screen.tsx`

Week 1: Screens and UI

Part A: Learn (Monday–Tuesday)

Monday

Concept 1.1 — How this project is organised

- What to learn: what `src/app`, `src/modules` and `src/shared` are; what a module and a feature are; where a screen's code lives.
- Read: README ("What the app does", "Project structure") · Feature reference ("What is a feature?")
- See it in the app: `src/app/(tabs)/products.tsx` → `src/modules/catalog/features/product-list/`
- Check yourself: if you add a new screen about products, which folder does its code go in?

Concept 1.2 — React components, props and lists

- What to learn: a component is a function that returns UI; props pass data in; `map` renders a list; every list item needs a `key`; `condition ? a : b` shows something conditionally.
- Read: Your first component · Passing props · Rendering lists · Conditional rendering
- See it in the app: `src/modules/catalog/features/product-list/product-card.tsx`
- Check yourself: what are the props of `ProductCard`?

Concept 1.3 — TypeScript for components

- What to learn: basic types (`string`, `number`, `boolean`), object types, optional props (`name?: string`), typing a component's props.
- Read: Everyday types · React + TypeScript cheatsheet ("Basic")
- See it in the app: the props type at the top of `product-card.tsx`
- Check yourself: how do you make a prop optional?

Tuesday

Concept 1.4 — React Native core components

- What to learn: `View` (a box), `Text` (all text must be inside it), `Pressable` (tappable), `ScrollView`, `Image`; opening other apps with `Linking` (email, phone).
- Read: Core components · Pressable · Linking
- See it in the app: `src/shared/components/settings-row.tsx`
- Check yourself: why can't you put a plain string directly inside a `View`?

Concept 1.5 — Layout and styling with theme colors

- What to learn: Flexbox (`flex-row` , `justify-between` , `items-center` , `gap-3`); NativeWind `className` ; theme tokens instead of hex colors; dark mode.
- Read: React Native: Flexbox · NativeWind docs · THEMING.md ("The tokens")
- See it in the app: the price row in `product-detail-screen.tsx`
- Check yourself: which class puts two items on one line, one left and one right?

Concept 1.6 — Data shapes with zod

- What to learn: a schema describes the data the API sends; `z.string()` , `z.number()` , `.optional()` , `.default()` , `z.array()` ; the TypeScript type comes from the schema.
- Read: Zod: basic usage · try `https://dummyjson.com/products/1` in your browser
- See it in the app: `src/modules/catalog/data/product.schema.ts`
- Check yourself: what happens if the API sends a field that isn't in the schema?

Concept 1.7 — Navigation with Expo Router

- What to learn: every file in `src/app` is a screen; folders become URLs; `[id]` is a dynamic part of the URL; `router.push(...)` opens a screen; the Stack in `_layout.tsx` .
- Read: Expo Router: core concepts · Navigating between pages · ARCHITECTURE.md ("Navigation")
- See it in the app: `src/app/products/[id]/index.tsx` and `src/modules/catalog/catalog.routes.ts`
- Check yourself: which file is shown for the URL `/products/5/edit` ?

Part B: Tasks (Wednesday–Friday)

Task 1.1 — "Contact support" row in Settings

- **When:** Wednesday.
- **Uses concepts:** 1.1, 1.4.
- **Objective:** let users email the support team directly from the app.
- **What you'll build:** a new row in Settings with a mail icon and the label "Contact support". Tapping it opens the phone's email app.
- **What to do:**
  1. Open `src/modules/settings/features/app-settings/settings-screen.tsx` .
  2. Copy the existing "UI Sandbox" `SettingsRow` . Change the icon to `Mail` and the label to "Contact support".
  3. On press, call `Linking.openURL('mailto:support@example.com')` .
- **Done when:** tapping the row opens the email app.

Task 1.2 — Rating stars component

- **When:** Wednesday.
- **Uses concepts:** 1.2, 1.3.

- **Objective:** show product ratings as 5 stars, the way shopping apps do, with one reusable component.
- **What you'll build:** a `<Rating value={4.6} />` component that shows 5 stars (filled up to the rating, empty after) and optionally the number.
- **What to do:**
  1. Create `src/modules/catalog/components/rating.tsx` with props `value: number` and `showValue?: boolean`.
  2. Render 5 stars with `Array.from({ length: 5 })`. A star is filled when its position is ≤ the rounded value. Look at how `product-detail-screen.tsx` draws one star.
  3. Use `<Rating />` in `product-card.tsx` instead of the current single star and number.
- **Done when:** every product card shows 5 stars, in light and dark mode.

Task 1.3 — "Details" card on the product screen

- **When:** Thursday.
- **Uses concepts:** 1.5, 1.6.
- **Objective:** show customers the warranty, shipping and return information before they buy.
- **What you'll build:** a card under the product description with three rows: Warranty, Shipping, Returns. Each row has the label on the left and the value on the right.
- **What to do:**
  1. Add `warrantyInformation`, `shippingInformation` and `returnPolicy` to `productSchema` as `z.string().optional()`.
  2. In `product-detail-screen.tsx`, add the card (`rounded-lg border border-border bg-card p-4`) below the description.
  3. Each row: `flex-row justify-between`; label `text-muted-foreground`, value `text-foreground`. Don't show a row whose value is missing.
- **Done when:** the card shows on product 1, looks right in dark mode, and `npm run validate` passes.

Task 1.4 — Reviews screen

- **When:** Friday.
- **Uses concepts:** 1.1, 1.6, 1.7.
- **Objective:** let customers read what other buyers said about a product.
- **What you'll build:** a "See all reviews" button on the product screen that opens a new screen listing every review: reviewer name, stars and comment.
- **What to do:**
  1. Add a `reviewSchema` (`rating`, `comment`, `reviewerName`) and `reviews: z.array(reviewSchema).default([])` to `productSchema`.
  2. Create the feature folder `src/modules/catalog/features/product-reviews/` with `product-reviews-screen.tsx` (takes `productId` as a prop, uses `useProduct(productId)`) and `index.ts`. Export the screen from `src/modules/catalog/index.ts`.

3. Create the route `src/app/products/[id]/reviews.tsx` (copy `src/app/products/[id]/edit.tsx`) and register it in `src/app/_layout.tsx` with the title "Reviews".
4. Add the button on the product screen. Use your `<Rating />` for each review.

- **Done when:** you can open the reviews of any product and go back.

Friday — weekly check

- [ ] 4 PRs open · `PROGRESS.md` updated · all screens checked in dark mode (Settings → Theme).

Week 2: Data — API, React Query and forms

Part A: Learn (Monday–Tuesday)

Monday

Concept 2.1 — How an app talks to a server

- What to learn: HTTP requests (GET to read, POST/PUT/DELETE to change), status codes (200, 404, 500), JSON.
- Read: MDN: An overview of HTTP · MDN: HTTP status codes · DummyJSON products API
- See it in the app: `src/shared/api/http-client.ts` (the only place that calls `fetch`)
- Check yourself: what does a 404 status mean?

Concept 2.2 — The layers: screen → hook → service → HTTP client

- What to learn: screens never call the API directly; a **service** calls the API and checks the answer with zod; a **hook** gives screens the data plus loading and error state.
- Read: ARCHITECTURE.md ("The layers inside a slice" and "READ: the list")
- See it in the app: `products.service.ts` → `product.queries.ts` → `product-list-screen.tsx`
- Check yourself: why does the service validate the response with zod?

Concept 2.3 — Reading data with React Query

- What to learn: `useQuery` gives `data`, `isPending`, `isError`, `refetch`; data is cached, so going back to a screen is instant; every screen shows loading, error, empty or data.
- Read: TanStack Query: overview · Queries · TkDodo: Practical React Query
- See it in the app: `useProduct` in `src/modules/catalog/data/product.queries.ts`
- Check yourself: what does the screen show while `isPending` is true?

Tuesday

Concept 2.4 — Query keys

- What to learn: the key is the cache "address" of the data; every value that changes the result (search, sort, id) must be part of the key.
- Read: Query keys · TkDodo: Effective React Query Keys
- See it in the app: `src/modules/catalog/data/product.keys.ts`
- Check yourself: what goes wrong if the search text is not in the key?

Concept 2.5 — Changing data with mutations

- What to learn: `useMutation` sends a POST/PUT/DELETE; after success, the cache is updated so every screen shows the new data.
- Read: Mutations · Updates from mutation responses
- See it in the app: `src/modules/catalog/features/edit-product/use-update-product.ts`
- Check yourself: after editing a product, why does the list update without a refresh?

Concept 2.6 — Forms with validation

- What to learn: react-hook-form holds the form's values; a zod schema checks them; errors show under each field.
- Read: React Hook Form: get started · useForm
- See it in the app: `src/modules/catalog/components/product-form.tsx` and `productFormSchema`
- Check yourself: where is the rule "Price must be greater than 0" written?

Concept 2.7 — Unit tests

- What to learn: a test runs a function and checks the result with `expect`; a "fake fetch" pretends to be the server.
- Read: Jest: getting started · expect
- See it in the app: `src/modules/catalog/data/__tests__/product.schema.test.ts`
- Check yourself: run `npm test`. How many tests pass?

Part B: Tasks (Wednesday–Friday)

Task 2.1 — Sort by price

- **When:** Wednesday.
- **Uses concepts:** 2.3, 2.4.
- **Objective:** let shoppers find cheap or premium products quickly.
- **What you'll build:** a button above the product list that switches between "Default", "Price: low to high" and "Price: high to low".
- **What to do:**
  1. In `product-list-screen.tsx`, keep the current sort in state and show a button that cycles through the three options.
  2. Pass `sortBy` and `order` into `useProductList(...)` and `productQueries.list(...)` in `product.queries.ts`. The service (`productsService.list`) already supports them.
  3. Add them to the key: `productKeys.list({ search, sortBy, order })`.
- **Done when:** the list re-orders when you tap the button, and search still works.

Task 2.2 — "Mark out of stock" action

- **When:** Wednesday.
- **Uses concepts:** 2.5.
- **Objective:** let a store manager mark a product as sold out in one tap.

- **What you'll build:** a "Mark out of stock" item in the menu that opens when you long-press a product. After tapping it, the product's badge says "Out of stock".
- **What to do:**
  1. Open `src/modules/catalog/features/product-list/product-actions-sheet.tsx` and add a new item, like "Edit product".
  2. In `product-list-screen.tsx`, handle it by calling `useUpdateProduct()` (import it from `../edit-product`) with the product's values and `stock: 0`. Payload: `productFormSchema.parse({ ...productToFormInput(product), stock: '0' })`.
- **Done when:** the badge changes to "Out of stock" immediately, in the list and on the detail screen.

Task 2.3 — Products by category screen

- **When:** Thursday.
- **Uses concepts:** 2.1, 2.2, 2.3, 2.7.
- **Objective:** let shoppers see all products of one category (for example all smartphones).
- **What you'll build:** a new screen `/products/category/[slug]` listing the products of a category, opened by tapping the category badge on a product's detail screen.
- **What to do:**
  1. Follow ADDING-A-FEATURE.md steps 1–9. It builds exactly this screen and includes the code (endpoint, key, service, hook, screen, route).
  2. Add the test for the new service function (step 2 of the recipe).
  3. Make the category badge on `product-detail-screen.tsx` tappable and navigate with `catalogRoutes.byCategory(product.category)`.
- **Done when:** tapping "smartphones" on a phone's detail page lists all smartphones, and `npm test` passes.

Task 2.4 — "Discount %" field in the product form

- **When:** Friday.
- **Uses concepts:** 2.5, 2.6, 2.7.
- **Objective:** let a store manager set a discount when creating or editing a product.
- **What you'll build:** a "Discount (%)" input in the product form that accepts 0–90.
- **What to do:**
  1. Add `discountPercentage` to `productFormSchema`: optional, a number from 0 to 90 (copy how `stock` turns text into a number).
  2. Add the input to `product-form.tsx` (copy the "Stock" field) with `keyboardType="number-pad"`, and add it to `productToFormInput` and `emptyProductForm`.
  3. In `product.schema.test.ts`, add a test that `95` is rejected.
- **Done when:** `95` shows an error, `20` saves and the product shows a 20% discount.

Friday — weekly check

- [ ] 4 PRs open · `PROGRESS.md` updated · `npm test` passes.

Week 3: Build your own module

Part A: Learn (Monday–Tuesday)

Monday

Concept 3.1 — Modules and their public API

- What to learn: a module is a business area (`catalog`, `users`); other code may only import its `index.ts`; ESLint blocks "deep" imports.
- Read: Module reference · ARCHITECTURE.md ("Module vs feature", "The import rules") · Jimmy Bogard: Vertical Slice Architecture
- See it in the app: `src/modules/catalog/index.ts`
- Check yourself: why can't the Home module import `@/modules/catalog/data/products.service`?

Concept 3.2 — Building a module, step by step

- What to learn: the order: endpoints → schema → service → query keys → queries → features → public API → routes.
- Read: ADDING-A-MODULE.md (the whole recipe) · DummyJSON users API
- Check yourself: which file is the only one that contains API paths?

Concept 3.3 — Testing a service with a fake fetch

- What to learn: replace `fetch` with a Jest mock, return a fake answer, check the result or the error.
- Read: Jest: mock functions · Jest: testing async code
- See it in the app: `src/modules/catalog/data/__tests__/products.service.test.ts`
- Check yourself: what does `mockFetchOnce(body, 404)` simulate?

Tuesday

Concept 3.4 — Lists and tabs

- What to learn: `FlashList` for long lists (`data`, `renderItem`, `keyExtractor`); adding a tab in `(tabs)/_layout.tsx`.
- Read: FlashList: usage · Expo Router: tabs
- See it in the app: `src/app/(tabs)/_layout.tsx`
- Check yourself: what does `keyExtractor` do?

Concept 3.5 — Dynamic routes and params

- What to learn: a route like `users/[id].tsx` reads `id` with `useLocalSearchParams` and passes it to the screen as a prop.

- Read: Expo Router: URL parameters
- See it in the app: `src/app/products/[id]/index.tsx`
- Check yourself: why does the route pass `productId` as a prop instead of the screen reading the URL itself?

Concept 3.6 — Login and storing a token safely

- What to learn: login sends username and password and gets back an **access token**; the token proves who you are on later requests; it must be stored securely, not in plain storage.
- Read: DummyJSON auth · Expo SecureStore
- Check yourself: why must a token never go in an `EXPO_PUBLIC_*` variable or plain storage?

Part B: Tasks (Wednesday–Friday)

Task 3.1 — Users data layer (with tests)

- **When:** Wednesday.
- **Uses concepts:** 3.1, 3.2, 3.3.
- **Objective:** give the app everything it needs to load users from the API, tested.
- **What you'll build:** a new `src/modules/users/` module with no screens yet: endpoints, schema, service, query keys, read hooks, public API, README.
- **What to do:**
  1. Follow ADDING-A-MODULE.md steps 1–5 and 8.
  2. Copy `products.service.test.ts` to `src/modules/users/data/__tests__/users.service.test.ts` and write 3 tests: the list loads, one user loads, a 404 throws an error.
  3. Add a short `README.md` (copy the shape of the catalog README).
- **Done when:** `npm run typecheck` and `npm test` pass.

Task 3.2 — Users tab

- **When:** Wednesday.
- **Uses concepts:** 3.4.
- **Objective:** show a list of everyone on the team.
- **What you'll build:** a new "Users" tab: a scrolling list with each person's photo, full name and email.
- **What to do:**
  1. Create `src/modules/users/features/user-list/` (screen + `index.ts`). Copy `product-list-screen.tsx` and remove what you don't need (search, sheets, delete).
  2. Each row: `Avatar` + name (`font-medium`) + email (`text-muted-foreground`).
  3. Add the route `src/app/(tabs)/users.tsx` and a tab with the `Users` icon.
- **Done when:** the Users tab scrolls through all users, with loading and error states.

Task 3.3 — User profile screen

- **When:** Thursday.
- **Uses concepts:** 3.5, 1.4.
- **Objective:** let people see a team member's details and contact them in one tap.
- **What you'll build:** a profile screen with a big photo, name, email, phone and city, and two buttons: **Call** and **Email**.
- **What to do:**
  1. Create `features/user-detail/` and the route `src/app/users/[id].tsx` (copy `src/app/products/[id]/index.tsx`). Register it in `src/app/_layout.tsx`.
  2. Add `usersRoutes.detail(id)` and open it when a row in the Users tab is tapped.
  3. Call: `Linking.openURL('tel:' + phone)`. Email: `Linking.openURL('mailto:' + email)`.
- **Done when:** tapping a user opens their profile, and Call opens the phone dialer.

Task 3.4 — Login

- **When:** Friday.
- **Uses concepts:** 2.6, 3.6.
- **Objective:** let a user sign in and stay signed in after closing the app.
- **What you'll build:** a login screen (username, password, "Sign in" button) opened from Settings. After signing in, Settings shows "Signed in as Emily" and a "Log out" button.
- **What to do:**
  1. Create a small `src/modules/auth/` module: schema for the login response, a service that calls `POST /auth/login`, and a login screen with a form (copy the pattern of `product-form.tsx`). Test user: `emilys` / `emilyspass`.
  2. Install SecureStore: `npx expo install expo-secure-store`. Save the `accessToken` after login; delete it on log out.
  3. In Settings, show the signed-in name and the Log out button, or a "Sign in" row.
- **Done when:** close and reopen the app and you're still signed in; Log out works.

Friday — weekly check

- [ ] 4 PRs open · `PROGRESS.md` updated · `npm run lint` passes (no deep imports).

Week 4: Polish and a mini project

Part A: Learn (Monday–Tuesday)

Monday

Concept 4.1 — Online and offline

- What to learn: the app can know when the phone is offline; React Query pauses requests and resumes them when the connection is back.
- Read: expo-network · TanStack: onlineManager
- See it in the app: `src/shared/lib/query-client.ts`
- Check yourself: what happens to a request started while offline?

Concept 4.2 — Shared components and the root layout

- What to learn: `src/shared/components` holds UI used everywhere; `src/app/_layout.tsx` wraps every screen, so something placed there shows on all screens.
- Read: Expo Router: layouts · src/shared/README.md
- Check yourself: why must code in `src/shared` never import from `src/modules`?

Concept 4.3 — Widgets: one module showing another's UI

- What to learn: a module can export a small self-contained widget (it loads its own data); another module places it on its screen.
- Read: Top-rated feature README
- See it in the app: `TopRatedCarousel` used in `home-screen.tsx`
- Check yourself: what does Home import, and from where?

Tuesday

Concept 4.4 — Screen states

- What to learn: every data screen handles 4 states: loading, error (with retry), empty, data.
- Read: Feature reference ("How it works")
- See it in the app: `src/shared/components/screen-states.tsx`
- Check yourself: what does the product list show when a search finds nothing?

Concept 4.5 — Accessibility

- What to learn: screen readers (VoiceOver, TalkBack) read `accessibilityLabel`; buttons need `accessibilityRole="button"`; touch targets should be big enough.
- Read: React Native: accessibility
- Check yourself: turn on VoiceOver or TalkBack and try the Products tab.

Concept 4.6 — Carts API

- What to learn: what the carts endpoints return (products, quantities, totals).
- Read: DummyJSON carts · open `https://dummyjson.com/carts/user/1` in your browser
- Check yourself: what is the difference between `total` and `discountedTotal`?

Part B: Tasks (Wednesday–Friday)

Task 4.1 — Offline banner

- **When:** Wednesday.
- **Uses concepts:** 4.1, 4.2.
- **Objective:** tell users clearly when they're offline, so they know why data isn't updating.
- **What you'll build:** a thin bar at the top of every screen: a `WifiOff` icon and "You're offline. Showing saved data." It appears only when the phone is offline.
- **What to do:**
  1. Create `src/shared/components/offline-banner.tsx` (`bg-muted`, `text-muted-foreground`).
  2. Use `useNetworkState()` from `expo-network` to decide whether to show it.
  3. Place it in `src/app/_layout.tsx`, above the screens.
- **Done when:** airplane mode on → the banner shows; off → it disappears.

Task 4.2 — "Team" widget on Home

- **When:** Wednesday.
- **Uses concepts:** 3.1, 4.3.
- **Objective:** show the team on the Home dashboard, using the users module's public API.
- **What you'll build:** a "Team" row on Home: 5 user photos side by side and a "See all" button that opens the Users tab.
- **What to do:**
  1. In `src/modules/users/features/`, create a `team-row` feature with a `TeamRow` component that loads users with `useUserList()` and shows the first 5 avatars.
  2. Export it from `src/modules/users/index.ts`.
  3. Show it in `src/modules/home/features/dashboard/home-screen.tsx`, importing only from `@/modules/users`.
- **Done when:** Home shows the Team row, and `npm run lint` passes.

Task 4.3 — Cart tab (mini project, part 1)

- **When:** Thursday.
- **Uses concepts:** 3.2, 3.4, 4.6. Do this one on your own: it's what Weeks 1–3 prepared you for.
- **Objective:** let the user see what's in their shopping cart.

- **What you'll build:** a `carts` module and a "Cart" tab that lists the products in user 1's cart: title, quantity and price for each line.
- **What to do:**
  1. Follow ADDING-A-MODULE.md with "carts" instead of "users": endpoint `GET /carts/user/1`, schema, service, keys, query.
  2. Create `features/cart-overview/` with the screen, and add the route `src/app/(tabs)/cart.tsx` with a `ShoppingCart` tab icon.
- **Done when:** the Cart tab lists the products in user 1's cart.

Task 4.4 — Cart summary and screen states (mini project, part 2)

- **When:** Friday.
- **Uses concepts:** 4.4, 4.5.
- **Objective:** make the cart screen complete and production-ready.
- **What you'll build:** a summary card at the bottom of the Cart tab (total, and the discounted total in `text-success`), plus loading, error and empty ("Your cart is empty") states.
- **What to do:**
  1. Add the summary card using `total` and `discountedTotal` from the API.
  2. Add `LoadingState`, `ErrorState` and `EmptyState` (copy how `product-list-screen.tsx` does it).
  3. Give every button and row an `accessibilityLabel`. Write a `README.md` for the carts module.
- **Done when:** the Cart tab works in light and dark mode, with all four states, and `npm run validate` passes.

Friday — final check

- [ ] All 16 PRs open · `PROGRESS.md` complete · `npm run validate` passes.

Calendar days 29–30: Look back

1. Record a **2-minute screen recording** of your app showing everything you added.
2. Rate yourself from 1 (not at all) to 5 (confident) in `PROGRESS.md`:
    - I can build a screen with the app's components and theme colors.
    - I can add a route and navigate to it.
    - I can load data with a schema, a service and a React Query hook.
    - I can change data with a mutation and see the screen update.
    - I can build a form with validation.
    - I can create a new module by following the recipe.
    - I can get unstuck on my own.

`PROGRESS.md` template

```
# Progress — <your name>

## Learning (Part A)

| Week | Concepts read (yes / partly) | My "Check yourself" answers (short) |
| ---- | ---------------------------- | ----------------------------------- |
| 1    |                              |                                     |
| 2    |                              |                                     |
| 3    |                              |                                     |
| 4    |                              |                                     |

## Tasks (Part B)

| Task | Name                | Status (done / partly / skipped) | PR link | Hours | Blockers
|
| ---- | ------------------- | -------------------------------- | ------- | ----- | --------
|
| 1.1  | Support row         |                                  |         |       |
|
| 1.2  | Rating stars        |                                  |         |       |
|
| 1.3  | Details card        |                                  |         |       |
|
| 1.4  | Reviews screen      |                                  |         |       |
|
| 2.1  | Sort by price       |                                  |         |       |
|
| 2.2  | Out-of-stock action |                                  |         |       |
|
| 2.3  | Category screen     |                                  |         |       |
|
| 2.4  | Discount field      |                                  |         |       |
|
| 3.1  | Users data layer    |                                  |         |       |
|
| 3.2  | Users tab           |                                  |         |       |
|
| 3.3  | User profile        |                                  |         |       |
|
| 3.4  | Login               |                                  |         |       |
|
| 4.1  | Offline banner      |                                  |         |       |
|
| 4.2  | Home Team widget    |                                  |         |       |
|
| 4.3  | Cart tab            |                                  |         |       |
|
| 4.4  | Cart summary        |                                  |         |       |
|
```

For the team lead: tracking progress

No teaching needed. Once or twice a week, look at each trainee's fork:

- **Task PRs:** 4 per week is on track.

- **`PROGRESS.md`:** concepts read, "Check yourself" answers, hours and blockers.

Trainee	Week 1	Week 2	Week 3	Week 4	Self-check avg	Still active?
	/4	/4	/4	/4	/5	yes / no

Good signs: a PR for every task, `npm run validate` passing, short honest answers in "Check yourself", and blockers written down instead of silence.

After the 4 weeks

Next topics: protected routes and token refresh, optimistic updates, component tests, performance, development builds and EAS (publishing), push notifications.

Links: Expo tutorial · React Native docs · TanStack Query · NativeWind · React Native Reusables